

Nanbeige4.1-3B: A Small General Model that Reasons, Aligns, and Acts

Nanbeige LLM Lab, Boss Zhipin

Abstract

We present Nanbeige4.1-3B, a unified generalist language model that simultaneously achieves strong agentic behavior, code generation, and general reasoning with only 3B parameters. To the best of our knowledge, it is the first open-source small language model (SLM) to achieve such versatility in a single model. To improve reasoning and preference alignment, we combine point-wise and pair-wise reward modeling, ensuring high-quality, human-aligned responses. For code generation, we design complexity-aware rewards in Reinforcement Learning, optimizing both correctness and efficiency. In deep search, we perform complex data synthesis and incorporate turn-level supervision during training. This enables stable long-horizon tool interactions, allowing Nanbeige4.1-3B to reliably execute up to 600 tool-call turns for complex problem-solving. Extensive experimental results show that Nanbeige4.1-3B significantly outperforms prior models of similar scale, such as Nanbeige4-3B-2511 and Qwen3-4B, even achieving superior performance compared to much larger models, such as Qwen3-30B-A3B. Our results demonstrate that small models can achieve both broad competence and strong specialization simultaneously, redefining the potential of 3B parameter models. The model checkpoint is available at <https://huggingface.co/Nanbeige/Nanbeige4.1-3B>

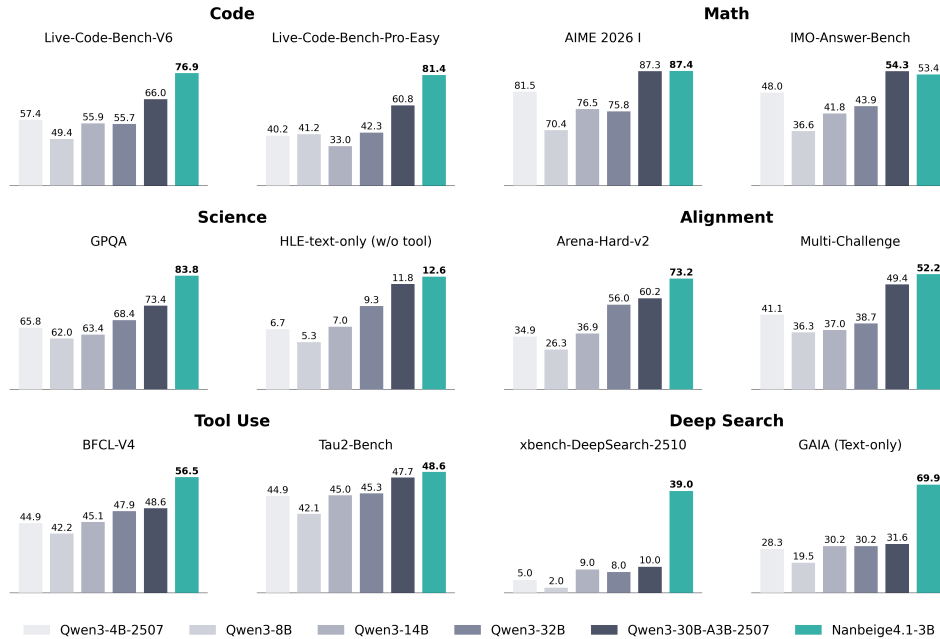


Figure 1: Performance of Nanbeige4.1-3B-Thinking vs. Qwen model series.

1 Introduction

Recent advances in small language models (SLMs) have demonstrated that compact models can achieve impressive performance on specialized tasks such as mathematical reasoning or code generation [21, 19]. Despite the small size, these models perform competitively by leveraging innovations in architecture, training data, and training algorithm, making them highly effective in practical applications.

However, most existing SLMs exhibit fragmented capabilities: reasoning-focused models often struggle with long-horizon interactions (e.g., deep search) [17], while code or agent specialized models typically lack robust general reasoning abilities, such as creative writing or human preference alignment [24]. Consequently, building a truly unified generalist model at the 3B scale remains an open challenge and a practical question is raised: **How far can a 3B model be pushed as a generalist without compromising its existing strengths?**

In this work, we present **Nanbeige4.1-3B**, a generic small language model that is remarkably powerful in reasoning, coding, and agentic behaviors, Built upon Nanbeige4-3B [21]. For general reasoning, we extend our previous pair-wise preference modeling by incorporating point-wise reward signals, ensuring that responses are both strong in isolation and preferred under direct comparison. For code generation, we move beyond correctness as the sole objective and explicitly reward algorithmic efficiency, encouraging solutions that are not only functionally correct but also computationally efficient. For agentic behavior, we emphasize long-horizon planning. We construct high-quality training data through Wiki-graph random walks and define rewards at both the interaction (turn) and full-trajectory levels, allowing the model to receive credit for planning and execution over hundreds of steps. Throughout training, we employ careful SFT data mixing and multi-stage reinforcement learning [15] to maintain balance across these domains.

Through the aforementioned approach, we have trained a model with an exceptionally broad and stable capability profile. Across reasoning and coding tasks, Nanbeige4.1-3B consistently surpasses existing open-source SLMs (e.g., Qwen3-4B and Qwen3-8B) and shows clear improvements over Nanbeige4-3B-Thinking-2511. More notably, Nanbeige4.1-3B exhibits deep-search and long-horizon agentic behavior rarely observed in general-purpose small language models. While models such as Qwen3-4B and Qwen3-8B fail to sustain meaningful exploration beyond a few turns, Nanbeige4.1-3B reliably solves complex search-oriented tasks through extended tool interaction. In this regime, a generalist 3B model attains deep-search capability comparable to that of specialized search-oriented models at the tens-of-billions scale, and approaches the search performance of 100B+ general-purpose large models. These results indicate that long-horizon agency can be achieved in compact generalist models when training objectives and credit assignment are properly aligned.

We open-source Nanbeige4.1-3B to support research into efficient, agent-capable language models. Beyond the checkpoint itself, we hope this release contributes to the community’s understanding of how to jointly train reasoning, coding, and long-horizon behaviors under strict capacity constraints.

2 Methods

In this section, we present the methodology used to equip a compact 3B model with broad generalist capabilities. We first describe how we individually optimize the model for general reasoning, long-horizon search (agentic behavior), and code generation, focusing on the design of training signals and reward structures tailored to each capability. We then detail how these heterogeneous objectives are integrated through data mixing and multi-stage training, enabling the model to retain domain-specific strengths while emerging as a unified general model under strict capacity constraints.

2.1 General Abilities

In this section, we elaborate on the optimization of Nanbeige4.1’s general capabilities. Our improvements focus on two key aspects: refining the data construction strategies in the SFT phase, and upgrading the General RL training paradigm through a progressive integration of Point-wise and Pair-wise reinforcement learning.

2.1.1 SFT

Nanbeige4.1-3B is built upon the Nanbeige4-3B-Base [21] with an enhanced SFT recipe, focusing on data distribution, length scaling, and training data quality.

First, we redesign the SFT data mixture. Compared to the previous Nanbeige4-3B-2511 version, we increase the proportion of code-related data and introduce a higher ratio of challenging problems in mathematics and general domains. This shift encourages stronger reasoning depth and improves robustness on difficult benchmarks.

Second, we extend the context length beyond the previous two-stage curriculum (32k $\rightarrow$ 64k) by introducing a third stage at 256k tokens to better support complex reasoning and long-horizon scenarios. In the final 256k stage, we adopt a specialized data mixture designed to strengthen agentic and reasoning capabilities, consisting of code (27%), deep-Search (26%), STEM (23%), tool-use (13%), and general domains (10%).

Third, we further optimize the Solution Refinement and Chain-of-Thought (CoT) Reconstruction framework originally introduced in Nanbeige4-3B-2511. Specifically, we scale up the number of refinement iterations in the Solution Refinement loop, allowing stronger critique–revision cycles to produce higher-quality final solutions. In addition, we train a more capable CoT Reconstruction model to generate cleaner and more faithful reasoning traces from refined answers.

As shown in Table 1, these improvements result in substantial gains across benchmarks. Nanbeige4.1-3B-SFT demonstrates consistent improvements in coding, mathematics, and alignment metrics, laying a stronger foundation for subsequent reinforcement learning stages.

Table 1: Performance Uplift from Nanbeige4-3B-SFT to Nanbeige4.1-3B-SFT

Domain	Benchmark	Nanbeige4-3B-SFT	Nanbeige4.1-3B-SFT	Δ
Code	LCB V6	45.5	62.0	+16.5
	LCB Pro Medium	1.8	22.8	+21.0
Math	Hmmt Nov	60.7	74.3	+13.6
	Imo-Answer-Bench	34.8	48.9	+14.1
Alignment	Arena-Hard V2	45.5	60.2	+14.7
	Multi-Challenge	42.6	44.4	+1.8

2.1.2 Point-wise RL

After SFT, we still observe some degradation issues in Nanbeige4.1-3B-SFT, such as repetition and redundant thinking, which have also been reported in prior work [5]. To address these issues and establish a more stable foundation for RL, we introduce a point-wise reinforcement learning stage.

We train a general reward model to evaluate rollout responses, following prior work [15]. The model is trained on curated large-scale human preference data. We found that the trained reward model naturally suppresses overly redundant, repetitive, and low-readability answers. We then perform GRPO [14] to optimize Nanbeige4.1-3B-SFT, sampling 8 rollouts per prompt and using the general reward model to score each response as the training signal.

With point-wise RL, we significantly reduce formatting errors and redundant reasoning. On LiveCodeBench-v6, the point-wise RL greatly improves length stability, reducing overlong truncation from 5.27% to 0.38%. As shown in Table 2, it also advances Arena-Hard V2 from 60.2 to 66.6, with the hard-prompt subset improving from 46.1 to 54.1. These gains are reflected in more consistent, well-structured outputs and high-quality code presentation.

2.1.3 Pair-wise RL

Although the Point-wise RL training provides effective alignment signals, the amount of high-quality preference data is limited, which constrains further improvement. To address this, we introduce Pair-wise RL to fully leverage preference information from strong–weak model comparisons and further enhance model performance.

We train a pair-wise reward model on paired comparison data spanning code generation and LMArena-style conversations (single-turn and multi-turn). We generate response pairs with a strong model and a weak model, then apply the same checklist filtering strategy as Nanbeige4 to derive reliable win-loss labels [21]. Following the framework [18], we mitigate position bias by adding a swap-consistency regularizer, defined as the mean squared error between the predicted reward difference for a response pair and the negated reward difference for the swapped pair.

We then run Pair-wise RL by formulating the reward as a binary outcome based on whether the generated rollout outperforms the reference answer. For multi-turn scenarios, we concatenate the full dialogue history into the pairwise reward model’s input.

As shown in 2, Nanbeige4.1-3B achieves comprehensive performance breakthroughs after Pair-wise RL. By deeply exploiting contextual information in multi-turn dialogues, Alignment metrics show significant gains, with Multi-Challenge increasing from 47.72 to 55.14. Additionally, Pair-wise RL notably boosts performance on the Arena-Hard V2 benchmark, with an improvement from 66.6 to 73.8, showcasing its effectiveness in refining alignment. These results confirm that Pairwise RL sharpens preference boundaries, providing the informative supervision signals needed for overall improvement across various benchmarks.

Table 2: Improvements via General RL Training (SFT → Point-wise RL → Pair-wise RL)

Domain	Benchmark	SFT	Point-wise RL	Δ	Pair-wise RL	Δ
Code	LCB V6	62.0	66.0	+4.0	65.6	-0.4
Alignment	Arena-Hard V2	60.2	66.6	+6.4	73.8	+7.2
	Multi-Challenge	44.4	47.7	+3.3	55.1	+7.4

2.2 Deep Search Ability

In this section, we primarily focus on enhancing the **deep search capabilities** of our model, specifically on the data pipeline and the staged training process. Deep search is defined as a retrieval-centric task that operates under complex multi-hop reasoning and extremely long-context settings. In this paradigm, models iteratively interact with the environment to acquire information, enabling them to solve challenging search problems.

2.2.1 Data Construction

To enhance the search capability of our base model, we construct a large-scale, complex search dataset, which includes a substantial number of multi-hop QA pairs derived from entity-relation graphs built upon Wikipedia, as well as high-quality long-range search trajectories that have undergone multi-stage rigorous filtering. The entire data construction pipeline is illustrated in Figure 2. To facilitate further research, we have open-sourced the constructed dataset on HuggingFace¹.

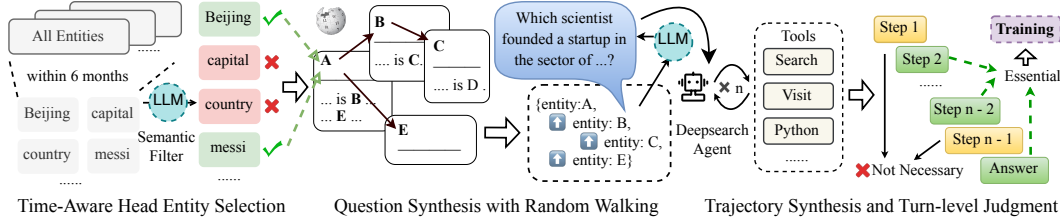


Figure 2: A data construction pipeline for deep search, including complex multi-hop QA sampling and the synthesis of long-horizon reasoning trajectories.

Temporal-Aware Head Entity Selection and Question Synthesis with Random Walking. To ensure the timeliness and complexity of the synthesized QA data, we first extract informative head

¹<https://huggingface.co/datasets/Nanbeige/ToolMind-Web-QA>

Table 3: Preliminary evaluation results on search benchmarks for the synthetic QA.

Benchmark	GAIA (text-only)	Browse Comp	Browse Comp-ZH	HLE (text-only)	SEAL-0	xBench DeepSearch-05	xBench DeepSearch-10
Nanbeige4-3B-2511	19.4	0.8	3.1	13.9	12.6	33.0	11.0
+ Synthetic QA	58.3	14.4	30.1	22.4	36.0	76.0	30.0

entities from Wikipedia that have been updated within the past six months. Following the framework of [20], we construct an entity-relation graph and perform conditional random walks to extract relational paths of a predefined length. These chains, along with their detailed temporal contexts, are then fed into a robust LLM to synthesize intricate questions.

Trajectory Synthesis and Turn-level Judgment. To synthesize search trajectories, we employ multiple agent frameworks to address the generated queries, sampling a diverse set of reasoning paths. These trajectories are subsequently mapped into multi-turn tool-invocation sequences from a unified agent perspective. To further guarantee the quality of the synthesized data, we implement a rigorous turn-level judgment mechanism. Specifically, we employ a critic model to evaluate each step of the interaction based on three dimensions: logical soundness, tool-call accuracy, and informational gain. Any turn that fails to meet these criteria does not participate in model training or provide a negative reward for the model. This fine-grained filtering ensures that the final trajectories provide a high-fidelity signal for supervised fine-tuning and preference alignment.

2.2.2 Preliminary Performance

To empirically validate the effectiveness of our proposed data construction pipeline, we conduct a controlled experiment using Nanbeige4-3B-2511 as the base model. Specifically, we train the model exclusively on the synthetic multi-hop QA and search trajectories generated via the methods described in Section 2.2.1, intentionally excluding other open-source data. We evaluate our model on a range of long-horizon benchmarks, including GAIA [9], BrowseComp [16], BrowseComp-ZH [23], Humanity’s Last Exam (HLE) [12], SEAL-0 [11], xBench-DeepSearch-2505 [3], and xBench-DeepSearch-2510 [3]. For HLE and GAIA, we only test and report the results of the text-only subset. In addition, HuggingFace has been explicitly disabled in these tools.

The model is evaluated within the Mindflow framework ², employing a suite of tools: Serper ³ for environment search, Jina ⁴ for webpage content extraction, and E2B Sandbox ⁵ as the secure sandbox environment. A comparative analysis of our results across specific stages is presented below:

The quantitative results are summarized in Table 3. The incorporation of our synthetic data yields a significant performance improvement across all benchmarks compared to the base model. Notably, the model achieves a substantial leap on xBench-DeepSearch-2505 (improving from 33.0 to 76.0), demonstrating that our data synthesis pipeline effectively endows the model with robust multi-hop reasoning and long-context search abilities.

2.3 Coding Ability

In this section, we primarily focus on enhancing the **coding capabilities** of our base model, including the data construction pipeline, staged training strategy, and evaluation settings.

2.3.1 Judge System

We build a unified **judge system** that is shared across SFT data construction, RL data construction, and subsequent RL training and evaluation. It combines a **multi-language sandbox** for execution-based correctness checking with a dedicated **instruct judge model** for time-complexity comparison. The instruct model is specifically trained for fast complexity assessment in RL settings, enabling

²<https://github.com/MiroMindAI/MiroThinker>

³<https://serper.dev/>

⁴<https://jina.ai/>

⁵<https://e2b.dev/>

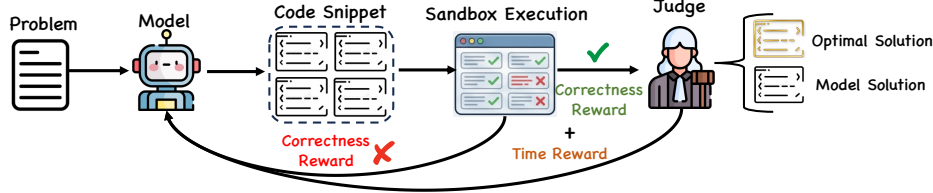


Figure 3: Gated time-complexity reward design in code RL. The time reward R_{time} is activated only when a solution passes all test cases (PassRate = 1), and the judge system provides online feedback by comparing the predicted time complexity against the reference optimal bound.

efficient online comparison between the predicted complexity of model-generated solutions and the reference optimal bound.

2.3.2 Data Construction

Our data construction consists of two components: SFT data construction with our judge system to filter time-optimal solutions, and RL data construction with on-policy difficulty filtering to improve sample efficiency.

SFT Data Construction. Our SFT data construction uses this judge system to assess solution quality from two key aspects: (1) *functional correctness* by executing the program in a sandbox, and (2) *time complexity* by combining execution signals with model-based complexity analysis. During data generation, we sample multiple candidate solutions per problem. The candidates are then verified by the judge system, and we keep those that are judged to be time-optimal (or among the best complexity class) for the given problem.

RL Data Construction. Each RL sample contains a problem statement, test cases, a time-complexity-optimal solution, and the corresponding optimal complexity label. The optimal solution and complexity are obtained by prompting multiple strong LLMs and employing a strong LLM to synthesize the candidates into a single best solution, which are then used as supervision signals for reward shaping and difficulty control.

In both stages, we perform **on-policy filtering** by running multiple rollouts per problem ($n = 8$) and selecting samples based on how many rollouts meet a stage-specific criterion. In Stage 1, we use a *difficulty-based* criterion: a problem is retained if the policy can solve it in a moderate number of rollouts (we keep problems with k in $[1, 5]$ successful solves out of 8). In Stage 2, we use a *complexity-based* criterion: we count how many rollouts produce solutions whose estimated time complexity satisfies the target bound, and retain problems with k in $[1, 5]$ complexity-satisfying rollouts out of 8.

2.3.3 Staged Training Process

Starting from the General-RL checkpoint, we further conduct two stages of code RL. In Stage 1, we optimize for solution correctness using a pass-rate reward, defined as the fraction of test cases passed for each problem. In Stage 2, after the policy can reliably solve problems, we additionally encourage higher-quality solutions by introducing a time-complexity reward *only when all test cases are passed*; otherwise the reward reduces to correctness-only signals. Specifically, the judge system provides online feedback by comparing the model’s output against the reference optimal complexity and checking whether the generated solution matches the reference optimal solution when applicable. As illustrated in Fig. 3, the time-complexity reward is activated only for fully correct solutions.

$$R = \begin{cases} R_{\text{format}} + R_{\text{correctness}}, & \text{PassRate} < 1, \\ R_{\text{format}} + R_{\text{correctness}} + R_{\text{time}}, & \text{PassRate} = 1. \end{cases}$$

As a concrete illustration, Appendix C presents LiveCodeBench case studies comparing model outputs **before** vs. **after** the time-reward stage, highlighting typical complexity-class improvements.

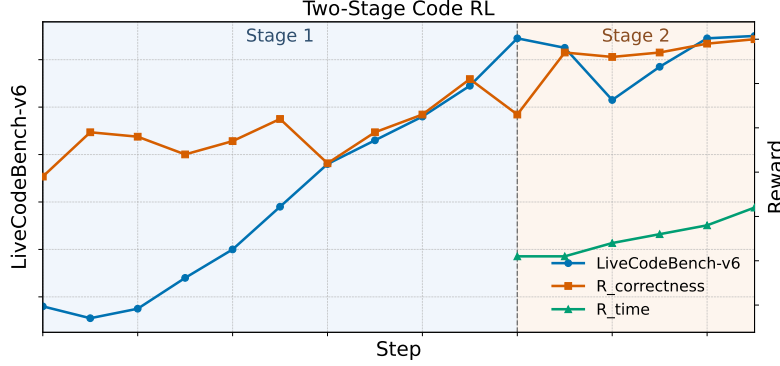


Figure 4: Training dynamics of two-stage code RL. We track the reward (including the gated R_{time} in Stage 2) and LiveCodeBench performance across training, showing consistent improvements from Stage 1 to Stage 2.

2.3.4 Training Dynamics

Across the two-stage code RL, we observe consistent improvements in both reward signals and downstream coding performance. In Stage 1, the *correctness reward* ($R_{\text{correctness}}$) increases sharply, reflecting rapid gains in producing valid and correct solutions. In Stage 2, $R_{\text{correctness}}$ improves more modestly, while the gated *time reward* (R_{time}) rises substantially, indicating that the policy is indeed optimizing time complexity once correctness is largely achieved. The overall reward and performance trends across the two-stage training are shown in Fig. 4.

2.4 Whole Training Recipe of Nanbeige4.1

Nanbeige4.1-3B is initialized from Nanbeige4-3B-Base and further optimized through a structured post-training pipeline combining large-scale SFT and cascaded RL.

We first conduct extended SFT, increasing the maximum context length from 64K to 256K compared to the previous Nanbeige4-3B-2511 version. This longer context window is essential for supporting long-horizon reasoning and multi-turn deep-search planning.

In the RL phase, we adopt a staged optimization strategy. General RL is performed sequentially with point-wise RL followed by pair-wise RL to enhance both standalone response quality and comparative preference alignment. Code RL is then conducted in two stages: a correctness stage that maximizes execution pass rate, followed by a gated time-complexity stage that activates efficiency rewards only when full correctness is achieved. Finally, we apply a lightweight agentic RL stage to strengthen tool-use and search behavior.

This unified training recipe enables Nanbeige4.1-3B to maintain strong domain-specific performance while emerging as a well-balanced generalist model under strict capacity constraints.

3 Experiment

Our evaluation comprises three components: general reasoning, deep-search agentic tasks, and a real-world coding challenge. General reasoning benchmarks measure core capability boundaries. Deep-search tasks evaluate long-horizon planning and tool-augmented multi-step reasoning in realistic environments. The real-world algorithm challenge provides an out-of-distribution stress test.

3.1 General Task Evaluations

For general reasoning capabilities, we evaluate across five major categories: **code**, **math**, **science**, **alignment**, and **tool-use**.

- **Code.** We report results on LiveCodeBench-V5, LiveCodeBench-V6 [6], and LiveCodeBench-Pro [22] to assess code generation ability and execution-based correctness under increasing difficulty.

Table 4: Evaluation results across code, math, science, alignment, and tool-use benchmarks.

Benchmark	Qwen3-4B 2507	Qwen3-32B	Qwen3-30B A3B-2507	Qwen3-Next -80B-A3B	Nanbeige4-3B 2511	Nanbeige4.1-3B
<i>Code</i>						
LCB-V6	57.4	55.7	66.0	<u>68.7</u>	46.0	76.9
LCB-Pro-Easy	40.2	42.3	60.8	<u>78.8</u>	40.2	81.4
LCB-Pro-Medium	5.3	3.5	3.5	<u>14.3</u>	5.3	28.1
<i>Math</i>						
AIME 2026 I	81.46	75.83	87.30	89.24	84.10	<u>87.40</u>
HMMT Nov	68.33	57.08	71.25	81.67	66.67	<u>77.92</u>
IMO-Answer-Bench	48.00	43.94	<u>54.34</u>	58.00	38.25	53.38
<i>Science</i>						
GPQA	65.8	68.4	73.4	77.2	<u>82.2</u>	83.8
HLE (Text-only)	6.72	9.31	11.77	13.70	10.98	<u>12.60</u>
<i>Alignment</i>						
Arena-Hard-V2	34.9	56.0	60.2	<u>62.3</u>	60.0	73.2
Multi-Challenge	41.14	38.72	49.40	56.52	41.20	<u>52.21</u>
<i>Tool Use</i>						
BFCL-V4	44.87	47.90	48.60	50.51	<u>53.80</u>	56.50
Tau2-Bench	45.90	45.26	47.70	57.40	41.77	<u>48.57</u>

- **Mathematics.** We include IMO-Answer-Bench [8], HMMT[1], and AIME-2026-I⁶ to evaluate symbolic reasoning and competition-level problem solving.
- **Science.** We benchmark on GPQA [13] and HLE [12] to measure multi-step scientific reasoning and domain knowledge integration.
- **Alignment.** We use Arena-Hard-V2 [7] and Multi-Challenge [4] to assess preference modeling robustness and response quality under adversarial or challenging prompts.
- **Tool-use.** We evaluate BFCL [10] and Tau2-Bench [2], which test function-calling reliability and multi-step tool use capability.

For compared baseline models, we include open-source models at similar scale (Qwen3-4B-2507) and our previous release (Nanbeige4-3B-2511), as well as substantially larger open models including Qwen3-30B-A3B-2507, Qwen3-32B, and Qwen3-Next-80B-A3B. This setup allows us to assess both same-scale competitiveness and cross-scale efficiency.

Overall Results. Table 4 shows that Nanbeige4.1-3B substantially outperforms both Qwen3-4B-2507 and its predecessor Nanbeige4-3B-2511 across all evaluated domains, demonstrating the effectiveness of our post-training strategy.

More notably, despite having only 3B parameters, Nanbeige4.1-3B consistently surpasses 30B–32B class models (Qwen3-30B-A3B-2507 and Qwen3-32B) on the majority of benchmarks, including coding, alignment, and tool-use tasks. The gains are particularly pronounced on execution-based coding benchmarks such as LiveCodeBench-V6 and LiveCodeBench-Pro-Medium, where Nanbeige4.1-3B achieves large absolute margins. When compared to Qwen3-Next-80B-A3B, Nanbeige4.1-3B remains competitive and exhibits complementary strengths: it leads on several coding and alignment benchmarks, while the 80B model retains advantages on selected mathematics and tool-use tasks.

These results suggest that strong cross-domain reasoning performance can be achieved through targeted post-training and agent-oriented optimization, even at a significantly smaller parameter scale.

⁶https://huggingface.co/datasets/MathArena/aime_2026_I

Table 5: Performance comparison across various deep search benchmarks.

Benchmark	GAIA (text-only)	Browse Comp	Browse Comp-ZH	HLE (text-only)	SEAL-0	xBench DeepSearch-05	xBench DeepSearch-10
Research Agents							
Tongyi-DeepResearch-30B	70.90	43.40	46.70	32.90	-	75.00	-
MiroThinker-v1.0-8B	66.40	31.10	40.20	21.50	40.40	60.60	-
AgentCPM-Explore-4B	63.90	25.00	29.00	19.10	40.00	70.00	-
Large Foundation Models with Tools							
GLM-4.6-357B	71.90	45.10	49.50	30.40	-	70.00	-
Minimax-M2-230B	75.70	44.00	48.50	31.80	-	72.00	-
DeepSeek-V3.2-671B	63.50	67.60	65.00	40.80	38.50	71.00	-
Small Foundation Models with Tools							
Qwen3-4B-2507	28.33	1.57	7.92	11.13	15.74	<u>34.00</u>	5.00
Qwen3-8B	19.53	0.79	5.15	10.24	6.34	31.00	2.00
Qwen3-14B	30.23	2.36	7.11	10.17	12.64	34.00	9.00
Qwen3-32B	30.17	3.15	7.34	9.26	8.15	39.00	8.00
Qwen3-30B-A3B-2507	31.63	1.57	4.12	<u>14.81</u>	9.24	25.00	10.00
Qwen3-Next-80B-A3B	<u>34.02</u>	<u>5.60</u>	<u>8.25</u>	9.26	<u>18.18</u>	27.00	6.00
Ours							
Nanbeige4-3B-2511	19.42	0.79	3.09	13.89	12.61	33.00	<u>11.00</u>
Nanbeige4.1-3B	69.90	19.12	31.83	22.29	41.44	75.00	39.00

3.2 Deep Search Task Evaluations

In this section, we present a comprehensive evaluation of the performance of Nanbeige4.1-3B on deep search tasks. We evaluate its capabilities in handling complex tasks by benchmarking against general foundation models of comparable size, specialized search-agent models, and significantly larger-scale foundation models.

3.2.1 Experimental Setup and Comparison Baselines

All experimental settings are the same as those of Section 2.2.2. These benchmarks require iterative retrieval, planning, tool interaction, and cross-document reasoning, reflecting realistic autonomous agent workloads. To thoroughly contextualize the performance of Nanbeige4.1-3B, we conduct a diverse comparison against several categories of existing models. This includes general-purpose foundation models equipped with tools (e.g., Qwen-series models), specialized search-agent models such as MiroThinker-8B, AgentCPM-Explore-4B, and Tongyi-DeepResearch-30B. We also incorporate large-scale open foundation models exceeding 100B parameters, comprising Minimax-M2.1, GLM-4.6, and DeepSeek-V3.2 for comparison.

3.2.2 Overall Performance Analysis

Table 5 presents a comprehensive performance comparison across various search agent benchmarks. While "Research Agents" (e.g., Tongyi-DeepResearch-30B) and "Large Foundation Models with Tools" (e.g., Minimax-M2, DeepSeek-V3.2) generally exhibit strong capabilities in their respective categories, particularly on GAIA and browsing tasks, the "Small Foundation Models with Tools" (Qwen3 series) typically show lower performance levels.

Our model, Nanbeige4.1-3B, demonstrates a remarkable leap in performance compared to its baseline Nanbeige4-3B-2511 and significantly outperforms other small foundation models with tools across all benchmarks. More critically, Nanbeige4.1-3B achieves state-of-the-art results across nearly all evaluated benchmarks, including GAIA (69.90), xBench-DeepSearch-05 (75.00), and SEAL-0 (41.44). These scores not only surpass its direct competitors in the small model category but also place it on par with, or even exceeding, the performance of many larger "Research Agents" and "Large Foundation Models with Tools." This robust performance underscores Nanbeige4.1-3B's exceptional effectiveness in complex search and agentic tasks, demonstrating that competitive capabilities can be achieved efficiently even with a smaller model size through our proposed methodology.

3.3 Real-world Algorithmic Challenges: LeetCode Weekly Contests

Beyond curated academic benchmarks, we further evaluate Nanbeige4.1-3B on real-world competitive programming tasks drawn from recent LeetCode weekly contests ⁷, providing a practical out-of-distribution stress test of algorithmic reasoning ability.

We apply Nanbeige4.1-3B, alongside Qwen models of varying scales, to solve contest problems under standard competitive programming settings. The generated solutions are directly submitted to the official LeetCode platform, and performance is measured by the final acceptance rate.

Table 6: Comparison of Pass Rates on LeetCode Weekly Contests 484–488.

Benchmark	Qwen3-4B-2507	Qwen3-32B	Qwen3-30B-A3B-2507	Nanbeige4.1-3B
LeetCode Pass Rate (%)	55.0	50.0	<u>65.0</u>	85.0

As shown in Table 6, Nanbeige4.1-3B successfully solves 17 out of 20 problems, achieving an overall pass rate of 85.0%. In virtual participation mode, where ranking is determined by correctness and time efficiency, our model achieves 1st place in Weekly Contest 487 and 3rd place in Weekly Contest 488. Compared to Qwen3 models of similar or substantially larger scales, Nanbeige4.1-3B demonstrates a clear performance advantage on these real-world contest tasks. Detailed problem statements and representative solution examples are provided in Appendix B.

4 Conclusion

In this work, we present Nanbeige4.1-3B, a unified generalist model that demonstrates advanced capabilities in reasoning, coding, and long-horizon search. We achieve this by integrating point-wise and pair-wise reward modeling for precise preference alignment, optimizing code generation for both correctness and computational efficiency, and incorporating both turn and trajectory level signals to enable robust long-horizon agentic planning. Consequently, Nanbeige4.1-3B consistently outperforms comparable small models and remains competitive with larger baselines across diverse benchmarks and real-world challenges.

Looking ahead, we aim to push the boundaries of compact models in complex coding and research agent scenarios. Concurrently, we focus on improving inference efficiency by enabling tasks to be solved with shorter outputs and fewer tool invocations, while exploring architectural innovations to further enhance the potential of the compact generalist model.

⁷<https://leetcode.cn/contest/>

References

- [1] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions. *arXiv preprint arXiv:2505.23281*, 2025.
- [2] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint*, 2025.
- [3] Kaiyuan Chen, Yixin Ren, Yang Liu, Xiaobo Hu, Haotong Tian, Tianbao Xie, Fangfu Liu, Haoye Zhang, Hongzhang Liu, Yuan Gong, Chen Sun, Han Hou, Hui Yang, James Pan, Jianan Lou, Jiayi Mao, Jizheng Liu, Jinpeng Li, Kangyi Liu, Kenkun Liu, Rui Wang, Run Li, Tong Niu, Wenlong Zhang, Wenqi Yan, Xuanzheng Wang, Yuchen Zhang, Yi-Hsin Hung, Yuan Jiang, Zexuan Liu, Zihan Yin, Zijian Ma, and Zhiwen Mo. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations, 2025.
- [4] Kaustubh Deshpande, Ved Sirdeshmukh, Johannes Baptist Mols, Lifeng Jin, Ed-Yeremai Hernandez-Cardona, Dean Lee, Jeremy Kritz, Willow E Primack, Summer Yue, and Chen Xing. Multichallenge: A realistic multi-turn conversation evaluation benchmark challenging to frontier llms. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 18632–18702, 2025.
- [5] Ruixiang Feng, Yuntao Wen, Silin Zhou, Ke Shi, Yifan Wang, Ran Le, Zhenwei An, Zongchao Chen, Chen Yang, Guangyue Peng, Yiming Jia, Dongsheng Wang, Tao Zhang, Lisi Chen, Yang Song, Shen Gao, and Shuo Shang. Pace: Prefix-protected and difficulty-aware compression for efficient reasoning, 2026.
- [6] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024.
- [7] Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. *arXiv preprint*, 2024.
- [8] Minh-Thang Luong, Dawsen Hwang, Hoang H Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, et al. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 35406–35430, 2025.
- [9] Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants, 2023.
- [10] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- [11] Thinh Pham, Nguyen Nguyen, Pratibha Zunjare, Weiyuan Chen, Yu-Min Tseng, and Tu Vu. Sealqa: Raising the bar for reasoning in search-augmented language models, 2025.
- [12] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, and et al. Humanity’s last exam, 2025.
- [13] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint*, 2023.
- [14] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

- [15] Boxin Wang, Chankyu Lee, Nayeon Lee, Sheng-Chieh Lin, Wenliang Dai, Yang Chen, Yangyi Chen, Zhuolin Yang, Zihan Liu, Mohammad Shoeybi, et al. Nemotron-cascade: Scaling cascaded reinforcement learning for general-purpose reasoning models. *arXiv preprint arXiv:2512.13607*, 2025.
- [16] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.
- [17] Sen Xu, Yi Zhou, Wei Wang, Jixin Min, Zhibin Yin, Yingwei Dai, Shixi Liu, Lianyu Pang, Yirong Chen, and Junlin Zhang. Tiny model, big logic: Diversity-driven optimization elicits large-model reasoning ability in vibethinker-1.5 b. *arXiv preprint arXiv:2511.06221*, 2025.
- [18] Wenyuan Xu, Xiaochen Zuo, Chao Xin, Yu Yue, Lin Yan, and Yonghui Wu. A unified pairwise framework for rlhf: Bridging generative reward modeling and policy optimization. *arXiv preprint arXiv:2504.04950*, 2025.
- [19] Shaotian Yan, Kaiyuan Liu, Chen Shen, Bing Wang, Sinan Fan, Jun Zhang, Yue Wu, Zheng Wang, and Jieping Ye. Distribution-aligned sequence distillation for superior long-cot reasoning. *arXiv preprint arXiv:2601.09088*, 2026.
- [20] Chen Yang, Ran Le, Yun Xing, Zhenwei An, Zongchao Chen, Wayne Xin Zhao, Yang Song, and Tao Zhang. Toolmind technical report: A large-scale, reasoning-enhanced tool-use dataset. *arXiv preprint arXiv:2511.15718*, 2025.
- [21] Chen Yang, Guangyue Peng, Jiaying Zhu, Ran Le, Ruixiang Feng, Tao Zhang, Wei Ruan, Xiaoqi Liu, Xiaoxue Cheng, Xiyun Xu, et al. Nanbeige4-3b technical report: Exploring the frontier of small language models. *arXiv preprint arXiv:2512.06266*, 2025.
- [22] Zihan Zheng, Zerui Cheng, Zeyu Shen, Shang Zhou, Kaiyuan Liu, Hansen He, Dongruixuan Li, Stanley Wei, Hangyi Hao, Jianzhu Yao, et al. Livecodebench pro: How do olympiad medalists judge llms in competitive programming? *arXiv preprint arXiv:2506.11928*, 2025.
- [23] Peilin Zhou, Bruce Leon, Xiang Ying, Can Zhang, Yifan Shao, Qichen Ye, Dading Chong, Zhiling Jin, Chenxuan Xie, Meng Cao, et al. Browsecomp-zh: Benchmarking web browsing ability of large language models in chinese. *arXiv preprint arXiv:2504.19314*, 2025.
- [24] Jialiang Zhu, Gongrui Zhang, Xiaolong Ma, Lin Xu, Miaosen Zhang, Ruiqi Yang, Song Wang, Kai Qiu, Zhirong Wu, Qi Dai, et al. Re-trac: Recursive trajectory compression for deep search agents. *arXiv preprint arXiv:2602.02486*, 2026.

A Author List

Authors are listed in **alphabetical order by first name**. Names marked with an asterisk (*) denote individuals who were previously affiliated with our team. Yang Song is the corresponding author and can be reached at songyang@kanzhun.com.

Chen Yang, Guangyue Peng, Jiaying Zhu, Ran Le, Ruixiang Feng, Tao Zhang, Xiyun Xu, Yang Song[†], Yiming Jia, Yuntao Wen, Yunzhi Xu, Zekai Wang, Zhenwei An*, Zhicong Sun, Zongchao Chen

B Leetcode Contest Examples

The following examples are drawn from LeetCode Weekly Contest 488 under virtual participation. Since the contest was conducted on the Chinese version of LeetCode, the original problem statements presented to the model were in Chinese, and the generated code contained Chinese comments. For clarity, we provide English translations of both the problem descriptions and the model’s comments below.

B.1 Problem Q1: Count Dominant Indices

Problem Description

Given: An integer array `nums` of length n .

Definition: An index i is called a *dominant index* if the following condition is met:

$$\text{nums}[i] > \text{average}(\text{nums}[i + 1], \text{nums}[i + 2], \dots, \text{nums}[n - 1])$$

Your task is to count the number of *dominant indices*.

Note:

- The average is the sum of the group divided by the number of elements.
- The rightmost element of the array is not considered a dominant index.

Model Output (Nanbeige4.1-3B):

The model correctly transformed the inequality to avoid floating-point precision issues and utilized a suffix sum approach to solve the problem in $O(n)$ time complexity.

```
1 from typing import List
2
3 class Solution:
4     def dominantIndices(self, nums: List[int]) -> int:
5         """
6         [Translated Comment] Count the number of dominant indices in the array.
7         Dominant Index Definition: Index  $i$  such that:
8          $\text{nums}[i] > \text{average}(\text{nums}[i+1], \dots, \text{nums}[n-1])$ 
9         Note: The rightmost element (index  $n-1$ ) is not considered a candidate.
10
11         Optimization Strategy:
12         1. Avoid floating-point arithmetic: Transform  $\text{nums}[i] > \text{sum} / \text{len}$ 
13            into  $\text{nums}[i] * \text{len} > \text{sum}$ .
14         2. Traverse from right to left, dynamically maintaining the suffix sum.
15         3. Time Complexity:  $O(n)$ , Space Complexity:  $O(1)$ .
16         """
17         n = len(nums)
18         # Boundary condition: No valid dominant index if length  $\leq 1$ 
19         if n <= 1:
20             return 0
21
22         count = 0
```

```

23     suffix_sum = 0 # Stores sum of nums[i+1] to nums[n-1]
24
25     # Traverse backwards from the second to last element to index 0
26     for i in range(n - 2, -1, -1):
27         suffix_sum += nums[i + 1] # Accumulate current right-side part
28         length = n - i - 1      # Number of elements on the right
29
30         # Strict inequality check (avoiding floating point issues)
31         if nums[i] * length > suffix_sum:
32             count += 1
33
34     return count

```

B.2 Problem Q2: Apply Operations to Maximize Array Sum

Problem Description

Given: An integer array `nums`.

Operation: Repeatedly perform the following until no more operations can be done:

- If there are two adjacent and equal elements in the array, select the **leftmost** such pair.
- Replace them with their **sum**.

The array size decreases by 1 after each merge.

Task: Return the final array after all possible merge operations are completed.

Model Output (Nanbeige4.1-3B):

The model employs a stack-based simulation to handle the "leftmost" merge constraint efficiently. This mimics the recursive merge process in a single pass with $O(n)$ time complexity.

```

1  from typing import List
2
3  class Solution:
4      def mergeAdjacent(self, nums: List[int]) -> List[int]:
5          """
6              [Translated Comment] Simulate the merge process using a stack:
7              - Traverse the array, comparing each element with the stack top.
8              - If equal, continue merging (pop stack top and accumulate) until
9                not equal or stack is empty.
10             - Push the merge result (or original element) onto the stack.
11             - The final stack content is the result after no more merges can be done.
12
13             Time Complexity:  $O(n)$ , each element is pushed/popped at most once.
14             Space Complexity:  $O(n)$ , max stack depth.
15          """
16          stack = []
17          for num in nums:
18              current = num
19              # Continuous merge: when stack top equals current, pop and accumulate
20              while stack and stack[-1] == current:
21                  current += stack.pop()
22              stack.append(current)
23          return stack

```

B.3 Problem Q3: Count Subarrays with Cost Constraint

Problem Description

Given: An integer array `nums` and an integer `k`.

Definition: The *cost* of a subarray `nums[l..r]` is defined as:

$$\text{cost} = (\max(\text{nums}[l..r]) - \min(\text{nums}[l..r])) \times (r - l + 1)$$

Task: Return the number of subarrays where the cost is less than or equal to `k`.

Model Output (Nanbeige4.1-3B):

The model correctly identifies this as a sliding window problem. To maintain the maximum and minimum values within the moving window efficiently, it utilizes two monotonic deques, achieving an optimal solution.

```
1  from collections import deque
2  from typing import List
3
4  class Solution:
5      def countSubarrays(self, nums: List[int], k: int) -> int:
6          # [Translated] Boundary check: empty array or negative k (cost is
6          ↪ non-negative)
7          if not nums or k < 0:
8              return 0
9
10         n = len(nums)
11         dq_max = deque() # Monotonic decreasing deque: stores indices of max
11         ↪ values
12         dq_min = deque() # Monotonic increasing deque: stores indices of min
12         ↪ values
13         l = 0
14         ans = 0
15
16         for r in range(n):
17             # Maintain max deque (pop back if <= current value)
18             while dq_max and nums[dq_max[-1]] <= nums[r]:
19                 dq_max.pop()
20             dq_max.append(r)
21
22             # Maintain min deque (pop back if >= current value)
23             while dq_min and nums[dq_min[-1]] >= nums[r]:
24                 dq_min.pop()
25             dq_min.append(r)
26
27             # Calculate cost for current window [l, r]
28             cur_max = nums[dq_max[0]]
29             cur_min = nums[dq_min[0]]
30             length = r - l + 1
31             cost = (cur_max - cur_min) * length
32
33             # Shrink left boundary: if cost > k, move l to the right
34             while cost > k and l <= r:
35                 # Remove left boundary influence on extremums
36                 if dq_max[0] == l:
37                     dq_max.popleft()
38                 if dq_min[0] == l:
39                     dq_min.popleft()
40                 l += 1
41             if l > r:
42                 break
43             # Recalculate cost after shrinking
```

```

44         cur_max = nums[dq_max[0]]
45         cur_min = nums[dq_min[0]]
46         length = r - l + 1
47         cost = (cur_max - cur_min) * length
48
49         # Accumulate all valid subarrays ending at r
50         if l <= r:
51             ans += (r - l + 1)
52
53     return ans

```

B.4 Problem Q4: Maximize Score of K Pairs

Problem Description

Given: Two integer arrays `nums1` (length n) and `nums2` (length m), and an integer k .

Task: Choose exactly k pairs of indices $(i_1, j_1), (i_2, j_2), \dots, (i_k, j_k)$ such that:

$$0 \leq i_1 < i_2 < \dots < i_k < n \quad \text{and} \quad 0 \leq j_1 < j_2 < \dots < j_k < m$$

The score for each pair (i, j) is `nums1[i] × nums2[j]`. Return the **maximum total score** (sum of products) achievable.

Model Output (Nanbeige4.1-3B):

The model proposes a Dynamic Programming approach using rolling arrays to optimize space. It constructs a 2D state transition '`curr[i][j]`' representing the max score selecting a specific number of pairs from the prefixes of both arrays.

```

1  from typing import List
2
3  class Solution:
4      def maxScore(self, nums1: List[int], nums2: List[int], k: int) -> int:
5          n, m = len(nums1), len(nums2)
6          # [Translated] Edge case: k=0 returns 0
7          if k == 0:
8              return 0
9          # Implicit constraint check n >= k and m >= k
10         if k > n or k > m:
11             return 0
12
13         # prev[i][j]: Max score choosing t-1 pairs from first i and first j
14         ↪ elements
15         prev = [[0] * (m + 1) for _ in range(n + 1)]
16         NEG_INF = float('-inf')
17
18         # Iterate to select 1 to k pairs
19         for _ in range(1, k + 1):
20             # curr initialized to -inf
21             curr = [[NEG_INF] * (m + 1) for _ in range(n + 1)]
22             for i in range(1, n + 1):
23                 num1 = nums1[i - 1]
24                 for j in range(1, m + 1):
25                     num2 = nums2[j - 1]
26                     # Three choices:
27                     # 1. Skip current element of nums1: state from curr[i-1][j]
28                     # 2. Skip current element of nums2: state from curr[i][j-1]
29                     # 3. Take current pair (i-1, j-1): state from prev[i-1][j-1] +
30                     ↪ product
31                     # (ensures strictly increasing indices)
32                     skip_i = curr[i - 1][j]

```

```

31         skip_j = curr[i][j - 1]
32         take = prev[i - 1][j - 1] + num1 * num2
33         curr[i][j] = max(skip_i, skip_j, take)
34     prev = curr # Rolling update: current layer becomes prev for next
                 ↪ round
35
36     # Final result: Max score choosing exactly k pairs from n and m elements
37     return int(prev[n][m])

```

C Time-Complexity Case Study

Scope and purpose. This section presents a small set of **time-complexity-focused case studies** comparing model outputs **before** vs. **after** introducing the *gated time reward* in code RL. Concretely, we use examples from **LiveCodeBench** where the two models produce functionally correct solutions but differ in their algorithmic efficiency, and we analyze how the time-reward stage encourages outputs closer to the time-optimal complexity class.

C.1 AtCoder ABC388 Excerpt: Simultaneous Kagamimochi

Problem Description (Excerpt)

Given: N mochi with non-decreasing sizes $A_1 \leq A_2 \leq \dots \leq A_N$.

Rule: You can stack mochi of sizes a (top) and b (bottom) iff $a \leq \frac{b}{2}$.

Task: Maximize the number K of kagamimochi that can be made simultaneously by choosing $2K$ mochi and pairing them.

Before (Candidate Code).

```

1  import sys
2
3  def main():
4      data = sys.stdin.read().strip().split()
5      if not data:
6          return
7      N = int(data[0])
8      A = list(map(int, data[1:1+N]))
9      # The input is already sorted, but we sort to be safe.
10     A.sort()
11     low = 0
12     high = N // 2
13     while low < high:
14         mid = (low + high + 1) // 2
15         ok = True
16         for i in range(mid):
17             if 2 * A[i] > A[N - mid + i]:
18                 ok = False
19                 break
20         if ok:
21             low = mid
22         else:
23             high = mid - 1
24     print(low)
25
26 if __name__ == "__main__":
27     main()

```

After (Candidate Code).

```

1 import sys
2
3 def main():
4     data = sys.stdin.read().strip().split()
5     if not data:
6         return
7     N = int(data[0])
8     A = list(map(int, data[1:1+N]))
9     # A is already sorted in non-decreasing order as per input specification.
10    i = 0
11    j = N // 2
12    count = 0
13    while i < N // 2 and j < N:
14        if A[i] * 2 <= A[j]:
15            count += 1
16            i += 1
17            j += 1
18        else:
19            j += 1
20    print(count)
21
22 if __name__ == "__main__":
23     main()

```

C.1.1 Complexity Analysis

Key observation. Because A is sorted, the problem reduces to matching a *small* mochi as the top with a *large* mochi as the bottom, under the constraint $2 \cdot A[\text{top}] \leq A[\text{bottom}]$, without reusing mochi.

Before: binary search over the answer. The before solution binary-searches K in $[0, \lfloor N/2 \rfloor]$. For each candidate $K = \text{mid}$, it checks feasibility by scanning $i = 0..K - 1$ and verifying $2A[i] \leq A[N - K + i]$.

- If we include the extra `sort()`, the time is $O(N \log N) + O(N \log N) = O(N \log N)$.
- If we use the input guarantee (already sorted) and drop `sort()`, the dominating part is the feasibility checks:

$$O\left(\sum_{\text{iterations}} \text{mid}\right) = O(N \log N),$$

because there are $O(\log N)$ iterations and each check costs $O(\text{mid}) \leq O(N)$.

- Space complexity is $O(N)$ for storing A .

After: two-pointers greedy matching. The after solution uses two pointers: i over the smaller half $([0, \lfloor N/2 \rfloor])$ and j over the larger half $([\lfloor N/2 \rfloor, N])$. Pointer j only moves forward, and i moves forward exactly when a valid pair is formed.

- Each pointer advances at most N times, so the total time is $O(N)$ (no sorting is needed).
- Space complexity remains $O(N)$ for the array.

Conclusion. Both solutions are correct, but after improves the core algorithmic cost from $O(N \log N)$ to $O(N)$ by eliminating the binary search and performing a single-pass greedy matching enabled by the sorted input.

C.2 AtCoder ARC195 A Excerpt: Twice Subsequence

Problem Description (Excerpt)

Given: A sequence $A = (A_1, \dots, A_N)$ and a target sequence $B = (B_1, \dots, B_M)$ with $1 \leq M \leq N \leq 2 \times 10^5$.

Task: Determine whether there exist *at least two* subsequences of A that match B . Two subsequences are considered different if they use different sets of positions in A , even if the resulting values are identical.

Before (Candidate Code).

```
1 import sys
2 import bisect
3 from collections import defaultdict
4
5 def main():
6     data = sys.stdin.read().strip().split()
7     if not data:
8         return
9     it = iter(data)
10    N = int(next(it))
11    M = int(next(it))
12    A = [int(next(it)) for _ in range(N)]
13    B = [int(next(it)) for _ in range(M)]
14
15    # Map each value to the sorted list of indices where it appears in A
16    pos_map = defaultdict(list)
17    for i, val in enumerate(A):
18        pos_map[val].append(i)
19
20    # Find the leftmost (earliest) matching subsequence
21    left = []
22    pos = -1
23    for b in B:
24        if b not in pos_map:
25            print("No")
26            return
27        lst = pos_map[b]
28        # Find first index > pos
29        idx = bisect.bisect_left(lst, pos + 1)
30        if idx == len(lst):
31            print("No")
32            return
33        i = lst[idx]
34        left.append(i)
35        pos = i
36
37    # Find the rightmost (latest) matching subsequence
38    right = []
39    pos = N
40    for b in reversed(B):
41        if b not in pos_map:
42            print("No")
43            return
44        lst = pos_map[b]
45        # Find last index < pos
46        idx = bisect.bisect_left(lst, pos)
47        if idx == 0:
48            print("No")
49            return
50        i = lst[idx - 1]
51        right.append(i)
```

```

52         pos = i
53     right.reverse() # restore order to match B
54
55     # If the two subsequences differ in at least one position, answer Yes
56     for l, r in zip(left, right):
57         if l != r:
58             print("Yes")
59             return
60     print("No")
61
62 if __name__ == "__main__":
63     main()

```

After (Candidate Code).

```

1  import sys
2
3  def main():
4      data = sys.stdin.read().split()
5      if not data:
6          return
7      it = iter(data)
8      N = int(next(it))
9      M = int(next(it))
10     A = [int(next(it)) for _ in range(N)]
11     B = [int(next(it)) for _ in range(M)]
12
13     # leftmost matching indices
14     left = []
15     j = 0
16     for i in range(N):
17         if j < M and A[i] == B[j]:
18             left.append(i)
19             j += 1
20             if j == M:
21                 break
22     if j < M:
23         print("No")
24         return
25
26     # rightmost matching indices
27     right = []
28     j = M - 1
29     for i in range(N-1, -1, -1):
30         if j >= 0 and A[i] == B[j]:
31             right.append(i)
32             j -= 1
33             if j < 0:
34                 break
35     right.reverse()
36
37     # compare left and right
38     for k in range(M):
39         if left[k] != right[k]:
40             print("Yes")
41             return
42     print("No")
43
44 if __name__ == "__main__":
45     main()

```

C.2.1 Complexity Analysis

Key idea: earliest vs. latest embedding. Let $L = (\ell_1, \dots, \ell_M)$ be the *leftmost* embedding of B in A , obtained greedily by scanning A left-to-right and matching $B_1, B_2, \dots$ as early as possible. Let $R = (r_1, \dots, r_M)$ be the *rightmost* embedding of B in A , obtained greedily by scanning A right-to-left and matching $B_M, B_{M-1}, \dots$ as late as possible.

If there exists at least one k such that $\ell_k \neq r_k$, then we can construct two different matching subsequences: one using the prefix choices from L up to some point, and another using the suffix choices from R (intuitively, there is “slack” at position k allowing a different choice of index while still completing the match). Conversely, if $L = R$ coordinate-wise, then every match is forced to use the same index for each B_k , so there is *exactly one* matching subsequence.

Before: value-to-positions map + binary search. The *before* approach preprocesses a dictionary `pos_map` mapping each value v to the sorted list of indices where $A_i = v$. Then:

- It constructs L by, for each B_k , binary searching the first occurrence index $> \ell_{k-1}$.
- It constructs R similarly from the right, binary searching the last occurrence index $< r_{k+1}$.
- Finally it checks whether L and R differ in any coordinate.

Time. Building `pos_map` costs $O(N)$. Each of the M steps performs one binary search on an occurrence list, costing $O(\log N)$ in the worst case. Total time is $O(N + M \log N)$ per test case.

Space. The map stores all indices once, so $O(N)$.

After: two-pointer greedy scans. The *after* approach eliminates the map and binary searches:

- Construct L with a single left-to-right scan of A using a pointer j over B .
- Construct R with a single right-to-left scan of A using a pointer j over B .
- Compare L and R ; output Yes if they differ, else No.

Time. Each scan is $O(N)$, and the final comparison is $O(M)$, hence total $O(N + M) = O(N)$ per test case.

Space. Arrays `left` and `right` store M indices, so $O(M)$ (and thus $O(N)$).

Conclusion. Both solutions rely on the same correctness criterion: there are at least two matching subsequences iff the leftmost and rightmost embeddings differ at some position. The *after* solution achieves optimal linear time by replacing the index-map + binary-search steps with two greedy linear passes, reducing the per-test-case complexity from $O(N + M \log N)$ to $O(N)$ while keeping $O(N)$ space.

C.3 AtCoder ARC194 Excerpt: Minimum Cost Sort

Problem Description (Excerpt)

Given: A permutation P of $(1, 2, \dots, N)$.

Operation: For $1 \leq i \leq N - 1$, swap adjacent elements (P_i, P_{i+1}) with cost i .

Task: Compute the minimum total cost to sort P in ascending order.

Before (Candidate Code).

```
1 import sys
2 import heapq
3
4 def solve():
5     data = sys.stdin.read().strip().split()
6     if not data:
7         return
```

```

8     it = iter(data)
9     N = int(next(it))
10    P = [0] + [int(next(it)) for _ in range(N)] # 1-indexed
11
12    # Initialize a min-heap with all adjacent inversions
13    heap = []
14    for i in range(1, N):
15        if P[i] > P[i + 1]:
16            heapq.heappush(heap, i)
17
18    total_cost = 0
19    while heap:
20        i = heapq.heappop(heap)
21        # Check if the inversion still exists (it might have been fixed already)
22        if not (P[i] > P[i + 1]):
23            continue
24        # Perform the swap
25        P[i], P[i + 1] = P[i + 1], P[i]
26        total_cost += i
27        # After swapping, check the three affected positions for new inversions
28        for idx in (i - 1, i, i + 1):
29            if 1 <= idx < N and P[idx] > P[idx + 1]:
30                heapq.heappush(heap, idx)
31
32    print(total_cost)
33
34 if __name__ == "__main__":
35     solve()

```

After (Candidate Code).

```

1  import sys
2
3  def solve() -> None:
4      input = sys.stdin.readline
5      N = int(input())
6      P = list(map(int, input().split()))
7
8      # Fenwick tree (1-indexed)
9      bit = [0] * (N + 2)
10
11     def update(i: int, delta: int) -> None:
12         while i <= N:
13             bit[i] += delta
14             i += i & -i
15
16     def query(i: int) -> int:
17         s = 0
18         while i > 0:
19             s += bit[i]
20             i -= i & -i
21         return s
22
23     total_cost = 0
24     for val in P:
25         # number of smaller elements to the left of 'val'
26         cnt = query(val - 1)
27         p0 = cnt + 1 # position of 'val' after larger elements to its left
28         ↪ have passed
29         if p0 < val:
30             terms = val - p0 # number of right moves needed
31             # sum from p0 to val-1 inclusive
32             total_cost += (p0 + val - 1) * terms // 2
33             update(val, 1)

```

```

33
34     print(total_cost)
35
36 if __name__ == "__main__":
37     solve()

```

C.3.1 Complexity Analysis

Before: heap-driven local bubble swaps. The `before` solution repeatedly fixes adjacent inversions, always picking the smallest index inversion via a min-heap, and then re-checks nearby positions.

- Each heap operation is $O(\log N)$.
- The number of swaps equals the inversion count of the permutation, which can be $\Theta(N^2)$ in the worst case (e.g., a reversed permutation).
- Moreover, indices may be pushed into the heap multiple times and later discarded, but the total number of heap pushes still scales with the number of performed swaps (constant-factor neighborhood updates per swap).
- Therefore, worst-case time complexity is $O(\text{inv}(P) \log N) = O(N^2 \log N)$, with $O(N)$ memory for the array and heap.

After: Fenwick tree (BIT) aggregation. The `after` solution processes values in input order and uses a Fenwick tree to count how many smaller elements have appeared so far. For each value `val`, it infers how far this element must move (in terms of adjacent swaps) in the stable insertion process, and accumulates the corresponding cost by a closed-form arithmetic progression.

- Each query and update is $O(\log N)$, performed once per element.
- Total time complexity is $O(N \log N)$ per test case.
- Space complexity is $O(N)$ for the BIT and input array.

Conclusion. Compared with the swap-simulating heap method that can degrade to $O(N^2 \log N)$, the BIT-based solution computes the minimum cost in $O(N \log N)$ by aggregating necessary movements and costs without explicitly simulating each adjacent swap.